

```
krishnamohan_koyya@instance-1:~/glarimy-odyssey/osfy-tdo-11/ums-search-service$ curl -X GET -i 'http://10.1
01.175.196:8080/user?phone=9731423166'
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 280
ETag: W/"118-UxutZNACKJYIOzjhdGkxgX72ZoE"
Date: Thu, 04 Aug 2022 04:36:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"size":3,"users":[{"name":"Krishna Mohan","phoneNumber":9731423166,"since":"2022-07-09T07:53:08.000Z"}, {"n
ame":"Krishna Mohan Koyya","phoneNumber":9731423166,"since":"2022-06-10T07:37:47.000Z"}, {"name":"KrishnaMoh
an","phoneNumber":9731423166,"since":"2022-07-09T08:04:30.000Z"}]}krishnamohan_koyya@instance-1:~/glarimy-o
dyssey/osfy-tdo-11/ums-search-service$
```

Figure 4: Results of the search API

```
name: ums-search-service
labels:
  name: ums-search-service
spec:
  type: LoadBalancer
  ports:
    - port: 8080
  selector:
    app: ums-search-service
```

The summary of the above manifest is that it creates a deployment, replica-set and service along with user-supplied configuration.

Assuming that the Kubernetes cluster with Kafka, Zookeeper, MySQL, *AddService* and *FindService* is already up and running along with Minikube Tunnel, deploy the above manifest.

```
kube create -f kube-deployment.yml
```

A successful deployment creates three pods for the *SearchService* as per the manifest. It can be verified using the following command.

```
kube get pods
```

The sample run gives the listing, as shown in Figure 1.

Since we also deployed a Kubernetes service endpoint for *SearchService*, its details can be found using the following command.

```
kube get services
```

The output looks as in Figure 2. Make a note of the IP addresses of various services to interact with them.

That's it! The *SearchService* is now up and running along with all other services on the cluster. Figure 3 presents the architecture of the application as of now.

Though the complete deployment looks sufficiently complex, you would agree with me that designing, developing and deploying these microservices individually is a lot simpler. That's the beauty of microservice architecture.

Let us just verify *SearchService* using the following command:

```
curl -X GET -i 'http://localhost:8080/user?phone=9731423166'
```

The results look something like what's shown in Figure 4.

## Other opportunities

We designed the UMS as a set of multiple microservices, and selected Kafka for messaging and MySQL for storage. We may use any other message broker like RabbitMQ and any other storage technology like MongoDB, Cassandra, etc.

On the other hand, we used three different platforms to develop three microservices. It is not necessary that it must always be a polyglot. You may use only one platform for all the services. You may also use any other platforms like Ruby, Go, etc. The point is that microservice architecture does not limit you to a specific set of tools or technologies.

However, observe that we are offering three different IP addresses to interact with three different services. Obviously, it doesn't make sense! We will explore patterns that are specifically applicable to microservices and connect all the dots in the next and final parts. **END** 🐧

### By: Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.